

Spire Deployment

- 1 General info..... 2**
 - 1.1 What is this guide for?.....2
 - 1.2 Kubernetes Overview.....2
- 2 Install Environment.....4**
- 3 Deploy Server.....5**
- 4 Deploy Agent.....7**
- 5 Deploy Workloads..... 9**

1 General info

1.1 What is this guide for?

This guide helps you set up a Kubernetes cluster in Docker Desktop. Within this cluster, you deploy a Spire server that acts as a Certificate Authority (CA) and a Spire agent that can attest to that server in order to get an x.509 SVID certificate. You will then deploy two workloads, which will contact the agent to receive their own x.509 SVIDs, which can then be used to set up an mTLS connection between them, where they both authorize each other using their certificates.

1.2 Kubernetes Overview

Kubernetes is a container orchestrator that maintains certain states for its resources. Essentially, this means that you define a resource, such as a server, and Kubernetes will maintain your defined state for the resource. Furthermore, this means that Kubernetes is essentially declarative, like SQL for example. This brings us to yaml-files, which are plaintext files where configurations can be defined, and these can then be loaded into Kubernetes. This is a common way to instruct Kubernetes and it is what we are mostly doing here. We have prepared all the necessary yaml-files, and this guide will contain the necessary commands to load them into kubernetes.

Let us imagine that the cluster is running, and now we want to create a workload (resource). Furthermore, let's imagine that the resource is a server. Here are things that we and Kubernetes need that you should know about.

First, the resource needs to have an image, which is the actual application. These can be pulled, or they can be local.

The resource will also need a "ConfigMap". This is essentially the core configuration of the resource. Generally, this is defined in a local file that is then loaded into Kubernetes. This is how we handle most resources in this demonstration.

When deploying a resource in Kubernetes, there are several ways for the resource to exist. For our server for example, we use what is called a "StatefulSet". This kind of deployment tells the cluster that this resource should be static. If the workload were to restart, we need it to have the same name for example, since other workloads might be connecting to it. By contrast, our other workloads use "Deployment" which does not require the same persistence.

When the resource has been deployed it becomes part of a default namespace. A namespace is a logical grouping inside Kubernetes. It is not essential to create a new namespace, but it is helpful to isolate the resources we are interested in.

The resource also needs a service account. This account will then be configured with permissions. For example, being able to ask the Kubernetes API server about other resources.

The service account does nothing on its own, it also needs defined roles assigned to it. These are assigned by defining roles and then binding those roles to the service account.

Basically anything else you can think of is defined inside a yaml file.

An abstract step-by-step rundown of what we actually do during this implementation:

1. We set up the cluster
2. We create the name space in our cluster
3. We create the service account and roles for our spire server
4. We deploy our spire server by loading the yaml-files (configurations)
5. Now we repeat step 2-4 but for the agent
6. We now use the python script files to create images that can be used by our workload
7. We deploy these workloads and they should now be able to communicate with each other.

2 Install Environment

This can be done in various environments, but we chose to do this with docker desktop. Some configurations or steps may vary with other environments. The following is our setup.

1. Install docker Desktop. Can be downloaded here:

<https://www.docker.com/products/docker-desktop/>

2. During installation, make sure to check “wsl2”.

3. Docker is not supposed to require a distro, but it did in our case, since docker would not start without it. You can run the following command to install Ubuntu:

```
wsl --install -d Ubuntu
```

4. When running Docker, go to settings and ensure both the following options are checked:

- “Use the WSL 2 based engine”
- “Use containerd for pulling and storing images”

5. You can now go to the Kubernetes-tab in the main menu, click to start the cluster and select “kubeadm”. “Kind” probably works as well, but this is what we did.

OBS

From here on, you will do everything in powershell. You need to have downloaded the files from GitHub, then open a powershell windows and cd to that folder.

3 Deploy Server

You have to CD to the root folder of the files in Powershell. The following are commands to be performed in order.

1. Create namespace

```
kubectl create namespace spire
```

This is the namespace (the logical grouping) that we use for the workloads (server, agent etc) that concern us in the cluster.

2. Create service account

```
kubectl create serviceaccount spire-server -n spire
```

This is the account that the server uses inside the cluster. We will later bind roles/permissions to this account.

3. Apply trustbundle config

```
kubectl apply -f spire-bundle-configmap.yaml
```

This creates a kubernetes object called ConfigMap. This will later be used by the server to store its trust bundle.

4. Apply role to service account

```
kubectl apply -f server-cluster-role.yaml
```

This is where we create the role with permissions, that we also bind to the service account.

5. Apply server configMap, statefulset, and service

```
kubectl apply -f server-configmap.yaml  
kubectl apply -f server-statefulset.yaml  
kubectl apply -f server-service.yaml
```

These are the ConfigMap for the server, containing internal configurations for the spire server. The StatefulSet is the instructions for kubernetes on how to run the container with the application. The service handles the internal communication with the server, such as dns and load balancing.

Checks

```
kubectl get statefulset --namespace spire  
kubectl get pods --namespace spire  
kubectl get services --namespace spire
```

These are checks to ensure everything is running as expected. It should look like the following image on the next page.

```
PS C:\Spire_k8s_cluster> kubectl get statefulset --namespace spire
```

NAME	READY	AGE
spire-server	1/1	10d

```
PS C:\Spire_k8s_cluster> kubectl get pods --namespace spire
```

NAME	READY	STATUS	RESTARTS	AGE
spire-agent-p6zv7	1/1	Running	0	10d
spire-server-0	1/1	Running	0	10d

```
PS C:\Spire_k8s_cluster> kubectl get services --namespace spire
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
spire-server	NodePort	10.100.135.192	<none>	8081:31453/TCP	10d

4 Deploy Agent

You have to CD to the root folder of the files in Powershell. The following are commands to be performed in order.

1. Create service account

```
kubectl create serviceaccount spire-agent -n spire
```

This creates the service account for the spire agent.

2. Apply role to service account

```
kubectl apply -f agent-cluster-role.yaml
```

As we did with the server, we create roles and bind them to the account.

3. Apply configmap

```
kubectl apply -f agent-configmap.yaml
```

The application configuration for the agent

4. Apply daemonset

```
kubectl apply -f agent-daemonset.yaml
```

The server used a StatefulSet, which is one way to tell kubernetes how to run the container. Here we use DaemonSet instead. A StatefulSet is used when you need persistence (which a server might need for static addresses and such). A DaemonSet is instead used to ensure that an agent runs on every node in the cluster. Apart from this, it is the same idea. It tells the cluster how to run the container.

Checks

```
kubectl get daemonset --namespace spire
kubectl get pods --namespace spire
```

These checks ensure everything is running as it should. It should look something like this:

```
PS C:\Spire_k8s_cluster> kubectl get daemonset --namespace spire
NAME          DESIRED   CURRENT   READY   UP-TO-DATE   AVAILABLE   NODE SELECTOR   AGE
spire-agent    1         1         1       1             1           <none>          10d
PS C:\Spire_k8s_cluster> kubectl get pods --namespace spire
NAME          READY   STATUS    RESTARTS   AGE
spire-agent-p6zv7  1/1     Running   0          10d
spire-server-0    1/1     Running   0          10d
```

This time, get pods will list both the server and the agent.

At this point, we can also confirm that the agent has received its SVID from the spire server.

Run the following:

```
kubectl logs -l app=spire-agent -n spire --tail=-1
```

You should see that it performs node attestation and receives its Spiffe ID (from the x.509, from the server). It should look something like this:

```
time="2026-05-12T11:25:27Z" level=info msg="SVID is not found. Starting node attestation" subsystem_name=attestor trust_domain_id="spiffe://nonanonymous.org"
time="2026-05-12T11:25:27Z" level=info msg="Node attestation was successful" reattestable=true spiffe_id="spiffe://nonanonymous.org/spire/agent/k8s_psat/spire-cluster/de776844-333e-4104-ae45-e1248b75eebf" subsystem_name=attestor trust_domain_id="spiffe://nonanonymous.org"
```

5 Deploy Workloads

You have to CD to the root folder of the files in Powershell. The following are commands to be performed in order.

1. Create node registration entry

```
kubect1 exec -n spire spire-server-0 -- /opt/spire/bin/spire-server entry create
-spiffeID spiffe://nonanonymous.org/ns/spire/sa/spire-agent -selector
k8s_psat:cluster:spire-cluster -selector k8s_psat:agent_ns:spire -selector
k8s_psat:agent_sa:spire-agent -node
```

This creates a registration entry on the spire server. A registration entry could be seen as a list item that the server keeps in order to determine which entities are allowed a Spiffe ID. This command is for the agent, which has already been granted its Spiffe ID at this point. What we do here specifically is that we define how that Spiffe ID should look. This is because the workloads need to reference their parent, as seen in later commands.

2. Create service account for each workload

```
kubect1 create serviceaccount workload1 -n default
kubect1 create serviceaccount workload2 -n default
```

3. Create entries for both workloads

```
kubect1 exec -n spire spire-server-0 -- /opt/spire/bin/spire-server entry create
-spiffeID spiffe://nonanonymous.org/ns/default/sa/workload1 -parentID
spiffe://nonanonymous.org/ns/spire/sa/spire-agent -selector k8s:ns:default
-selector k8s:sa:workload1
```

```
kubect1 exec -n spire spire-server-0 -- /opt/spire/bin/spire-server entry create
-spiffeID spiffe://nonanonymous.org/ns/default/sa/workload2 -parentID
spiffe://nonanonymous.org/ns/spire/sa/spire-agent -selector k8s:ns:default
-selector k8s:sa:workload2
```

Again, these are registration entries for the server. This tells the server to create a Spiffe ID for a workload and how it should look. It also passes the parent ID that we defined in step 1.

4. Build images from the py-files

```
docker build -t workload1:v1 -f images/Dockerfile.workload1 images/
docker build -t workload2:v1 -f images/Dockerfile.workload2 images/
```

We now use the supplied python files to build actual images (applications). If you look in the images folder, you will see that there is a requirements.txt. This contains the name of the Spiffe libraries for python.

5. Deploy Workload1 and add service

```
kubectl apply -f workload1.yaml  
kubectl apply -f workload1-service.yaml
```

We deploy the workload (workload1) which is configured to use the image that we just created. We then also set up a service. Workload2 will be the one initiating contact with workload1.

6. Deploy Workload2

```
kubectl apply -f workload2.yaml
```

We now deploy this workload.

Checks

```
kubectl logs -l app=workload1  
kubectl logs -l app=workload2
```

The python scripts will make these two workloads communicate over mTLS, where they both verify each other's SVID certifications using the Spiffe libraries. They are programmed to print each other's responses, which are also logged. They run infinite loops where they communicate every 5th second. The logs should look something like this:

```
PS C:\Spire_k8s_cluster> kubectl logs -l app=workload1  
2026-05-12 11:17:48 - Received mTLS connection from workload 2: Hello from workload2  
2026-05-12 11:17:53 - Received mTLS connection from workload 2: Hello from workload2
```

At this point, you can also run the log command for the spire agent again:

```
kubectl logs -l app=spire-agent -n spire --tail=-1
```

It will show you various things about how the workload attests to the agent, and how the agent inspects the process of the workload.